TDD – Desarrollo dirigido por pruebas:

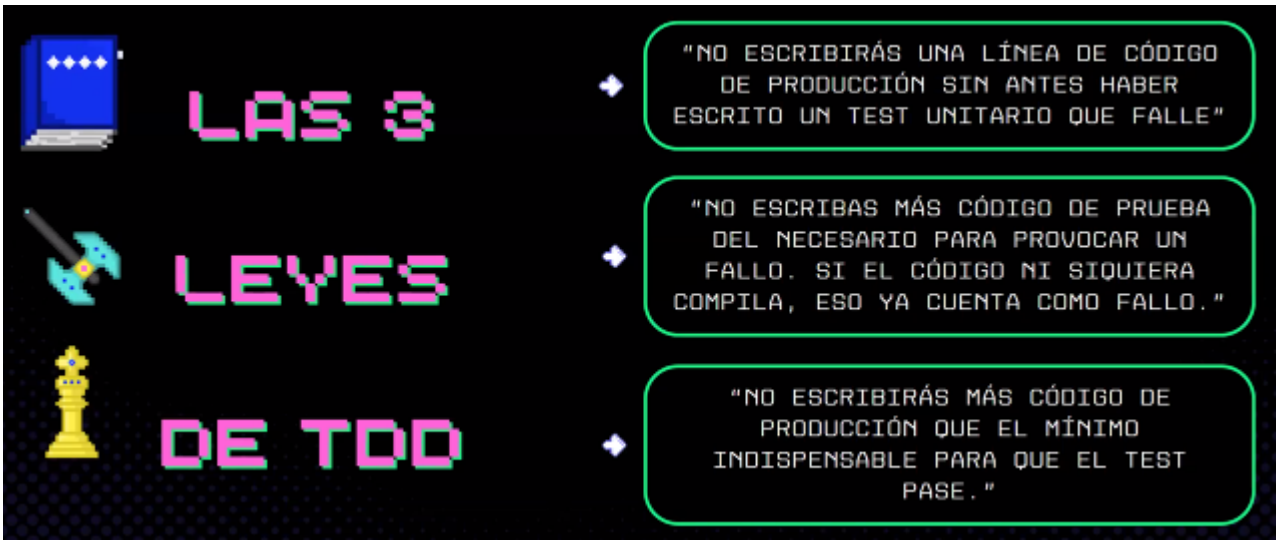
- Los desarrolladores, desarrollaban creyendo que todo iba a funcionar sin tener en cuenta partes aisladas
- La estructura es frágil
- El costo no es cte a medida que vas desarrollando incrementa resolver bugs son cada vez mas caros

Acoplamiento incidental
Deuda técnica y miedo al cambio
Costo asintótico del error

Estos ítems provocan que se cree el TDD, nuevo paradigma, no es una práctica de testing es practica de desarrollo que utiliza el testing para desarrollar
TDD es una actividad de especificación y diseño que guía el desarrollo por medio de test unitario, definiendo la meta para luego el como

Primero se define la prueba para dsp desarrollar lo que voy a probar

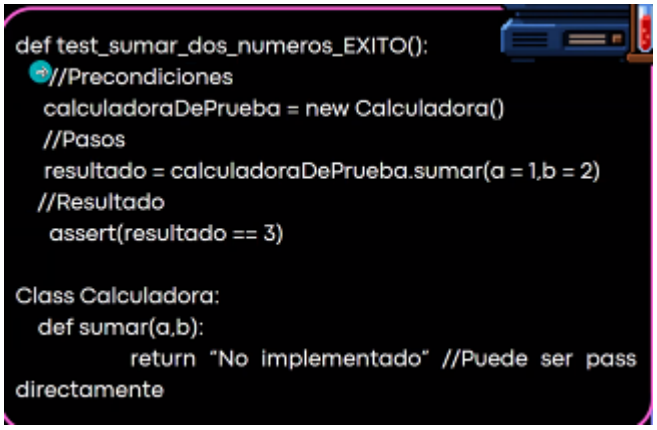
TRES LEYES DE TDD



CICLO RED GREEN REFACTOR

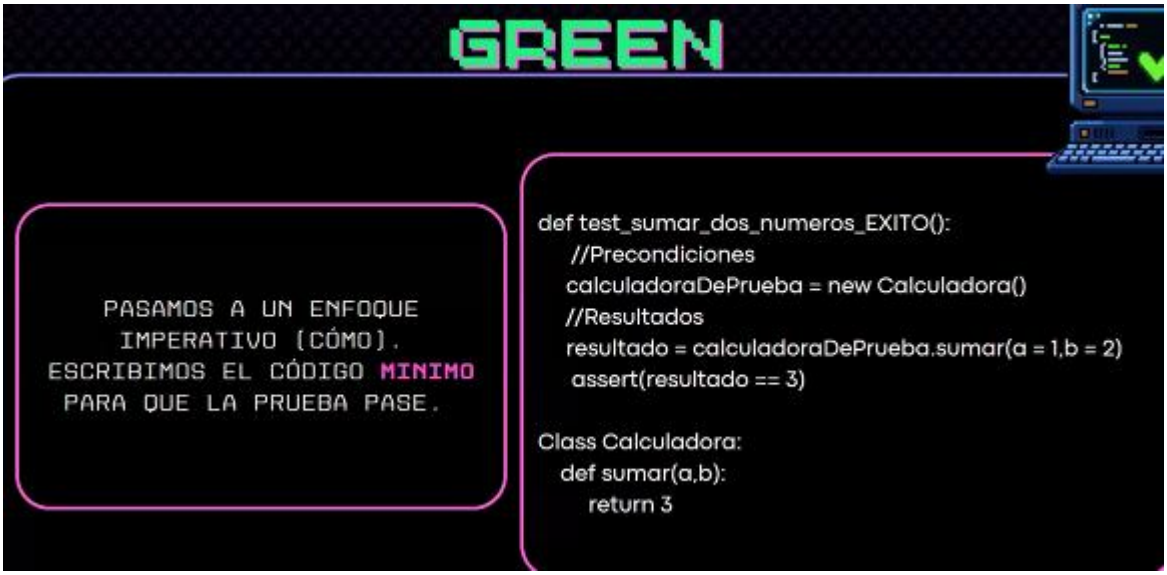
Ciclo repetitivo
RED: Se define una prueba que falle, el objetivo de la función es que mediante ciertos datos se obtiene cierto resultado (que falle), se declara lo que se quiere hacer
GREEN: (debería pasar la prueba del red)

Formato del Test:



RED

Patron fake it



En TDD tenemos que avanzar paso a paso, esto seria el pasito poner a+b seria la forma general

Refactory : Refactorisar la estructura interna sin afectar al resto del sistema, no es reestructuración a nivel de lógica si no a buenas practicas

Patrones para RED:
Starter Test = punto de partida para comenzar el desarrollo de pruebas, el camino mas obvio, destraba el resto de aristas, elegir mas trivial, mas obvio para arrancar el ciclo

One step test = una vez que arranque segui con el siguiente nivel de complejidad

Explanation test = etapa madura del ciclo, permite que al trabajar en equipo surgen debate en el comportamiento del sistema, dicho debate expresarlo ccomo test

Learning test = cuando quiero conectarme con un sist externo, librería o algo que no es un código interno, expresarlo como otro test, para entender como funciona y luego implementarlo

Another test = no entendí / noi escuche

Regretion test = Cuando ocurre un problema en tempo de ejecución voy a definir un test que defina los pasos que nos llevaron a ese error

Patrones para GREEN:
Fake it: Hardcodeas el retorno para pasar el test

Triangulación: Seguido del fake it, armar varios casos de fallas, para determinar que la falla esta en la lógica, ¿Seria hacer una falla genérica?

Obvious implementación: Si estas seguro que la solución es lo que decis no es necesario se purista con la practica RED GREEN REFACTOR, directamente escribis la solución general
(hace falta los test pero no hace falta hacer muchos ciclos)

One to many: pensamiento sistemico,

Practicas de diseño de test:
Test list: a

Isolated test: característica mas que patron, prueba unitaria

Test first: va de la mano de la primera ley, primero escribis la prueba

Assert test: cuando me cuesta escribir un test por la problemática, arranca desde el resultado y para obtener ese resultado necesito estos pasos, y para estos pasos necesito este entorno

Test data:

Evident data: Si se tiene un calculo complejo, la relación entre entrada y salida debe ser obvia

